

CTI ADP Software: Testing Strategy

Version: 1.0 | Updated: 2026-05-04 | Status: Active

1. TEST CONTEXT

Projects Covered

Project	Type	Tech Stack	Key Features
E-Commerce Frontend	Web App	React 18, TypeScript, Vite	Cart management, product browsing, checkout
E-Commerce Backend	REST API	Express.js, TypeScript, DDD	User auth, product catalog, order management
Mobile App	Android	Kotlin, Jetpack Compose, WebView	Display eCommerce in WebView
Desktop App	Desktop UI	Java 17, JavaFX, PostgreSQL	Stock management system

Purpose

- Ensure correctness across multi-platform application
- Prevent regressions in shared services (backend API)
- Validate platform-specific implementations (mobile/desktop UI)
- Support continuous integration and deployment

2. SCOPE

In Scope (What We Test)

E-Commerce Frontend:

- CartContext (add, remove, update, clear operations)
- localStorage persistence & recovery
- Stock validation & quantity constraints
- Component interactions (add-to-cart, checkout flows)

E-Commerce Backend:

- Use cases (CreateOrder, GetProducts, Authentication, etc.)
- Controllers (Auth, Product, Order endpoints)
- Security middleware & JWT validation
- PasswordService & JwtService

Mobile App:

- WebView loading & navigation
- Navigation between screens
- WebView-native communication
- Compose UI components

Desktop App:

- Stock management CRUD operations
- User authentication (login/register)
- Database persistence (PostgreSQL)
- Scene/UI state management

Out of Scope (What We Don't Test)

- E2E browser automation (Playwright/Cypress) — manual testing used
- Performance/load testing — future phase
- Accessibility (a11y) testing — future phase
- Third-party library internals
- Database migrations

3. STANDARD

Coverage Targets

Layer	Target	Tool
Backend Use Cases	90%+	Jest
Backend Controllers	85%+	Jest
Backend Middleware	90%+	Jest
Frontend Context Logic	90%+	Vitest
Frontend Components	75%+	Vitest
Mobile Compose UI	70%+	Espresso
Desktop Controllers	80%+	JUnit5

Quality Standards

- **Type Safety:** No any types; 100% TypeScript compliance
- **Test Naming:** Clear format: {unit} should {behavior} when {condition}
- **Code Quality:** All tests pass locally before CI, ESLint/type-check pass
- **Isolation:** Each test independent; no shared state between tests
- **Pass Criteria:** All assertions pass, coverage meets targets, no flaky tests

4. METHODOLOGY

Testing Approach by Layer

Local Development

1. Developer runs tests locally: `npm test` , `gradle test` , `mvn test`
2. Type checking: `tsc --noEmit` (TypeScript projects)
3. Linting: `eslint` or `checkstyle` based on project
4. Only commit if all pass

CI/CD Pipeline

1. Tests run automatically on every PR
2. Coverage report generated and commented
3. Must pass before merge approval
4. Flaky tests investigated immediately

Manual Testing

- Post-merge: browser testing of complete workflows
- Staging environment validation
- User story verification against acceptance criteria

5. TEST TYPES

Type	Definition	Coverage
Unit	Individual functions/components with mocked dependencies	Business logic, utilities, state calculations
Integration	Components with real dependencies (localStorage, DB calls mocked)	Context hooks, service layers, controllers
Component	React/Compose/JavaFX UI with user interactions	Buttons, forms, navigation, state updates
API Contract	Verify API responses match expected schema	Endpoint structures, data types, status codes
Error Scenario	Edge cases and error handling paths	Stock limits, auth failures, validation errors

Examples by Project

E-Commerce Backend (Jest):

```
// Unit: Test LoginUseCase with mocked repository
// Integration: Test AuthController with request/response cycle
// Error: Test JWT validation failure, invalid credentials
```

E-Commerce Frontend (Vitest):

```
// Unit: Test CartContext calculations (totalItems, totalAmount)
// Integration: Test useCart with CartProvider and localStorage
// Component: Test AddToCartButton with user interactions
```

Mobile (Espresso):

```
// Component: Test WebView loads and navigation works
// Integration: Test screen transitions
// UI: Test Compose button clicks and state updates
```

Desktop (JUnit5):

```
// Unit: Test ProductService business logic with mocked DB
// Integration: Test ProductController with HikariCP connection pool
// Error: Test invalid product creation, duplicate stock entries
```

6. TOOLS

E-Commerce Frontend

Tool	Version	Purpose
Vitest	^4.1.1	Test runner (Vite-native)
@testing-library/react	^16.3.2	Component testing
@testing-library/user-event	^14.5.2	User interaction simulation
jsdom	^29.0.1	DOM environment

Run: npm test , npm run test:coverage , npm run test:ui

E-Commerce Backend

Tool	Version	Purpose
Jest	^29.7.0	Test runner & assertions
ts-jest	^29.1.1	TypeScript support
jest-mock-extended	latest	Deep mocking

Run: `npm test` , `npm run test:watch` , `npm test -- --coverage`

Mobile (Android)

Tool	Version	Purpose
JUnit4	4.13.2	Local unit tests
Espresso	3.5.1	UI automation tests
Compose UI Test	latest	Jetpack Compose testing
AndroidJUnitRunner	1.1.5	Instrumented test runner

Run: `./gradlew test (unit)`, `./gradlew connectedAndroidTest (UI)`

Desktop (Java)

Tool	Version	Purpose
JUnit5 (Jupiter)	5.9.2	Test runner & assertions
Mockito	5.2.0	Mocking framework
Byte Buddy	1.14.10	Mock maker (Java 17+ compatible)

Run: `mvn test` , `mvn test -Dtest=ClassName`

7. INPUT DATA REQUIREMENTS

Test Fixtures (Reusable Data)

E-Commerce Frontend

```
const mockProduct = {  
  id: 1, sku: "PROD-001", name: "Test Product",  
  onhand: 10, price: 99.99, image_url: "...",  
};  
  
const mockCartItem = { product: mockProduct, quantity: 2 };
```

E-Commerce Backend

```
const validUser = {  
  email: "test@example.com", password: "SecurePass123!",  
};  
  
const product = {  
  id: 1, sku: "PROD-001", name: "Laptop", onhand: 5,  
  price: 999.99,  
};
```

Mobile

```
val mockProduct = Product(  
  id = 1, name = "Test Product", price = 99.99  
)  
  
val mockCart = listOf(  
  CartItem(product = mockProduct, quantity = 2)  
)
```

Desktop

```
Product mockProduct = new Product(  
    1, "PROD-001", "Laptop", 5, 999.99  
);  
  
User mockUser = new User(  
    1, "john@example.com", "hashedPassword", "John Doe"  
);
```

Mock Setup

Frontend (localStorage):

- Automatically cleared between tests (jsdom)
- Use `localStorage.setItem()` in test setup

Backend (Database):

- Mock Prisma repositories
- Use `jest.mock()` for external APIs

Mobile (WebView):

- Mock WebView callbacks
- Use Robolectric for context mocking

Desktop (Database):

- Mock HikariCP connection pool
- Use Mockito for service dependencies

Quick Reference

Run All Tests

```
# Frontend
cd e-commerce/frontend && npm test

# Backend
cd e-commerce/backend && npm test

# Mobile
cd mobile && ./gradlew test

# Desktop
cd desktop && mvn test
```

Coverage Reports

```
# Frontend: coverage/index.html
npm run test:coverage

# Backend: coverage/index.html
npm test -- --coverage --coverageReporters=html

# Desktop
mvn jacoco:report
```

Debug Tests

```
# Frontend (Interactive UI)
npm run test:ui

# Backend (VSCode Debugger)
node --inspect-brk ./node_modules/.bin/jest --runInBand
```

Last Reviewed: 2026-05-04

Next Review: 2026-06-04